

Interview guide for iDSP

Machine Coding

Objective

Competencies to Evaluate

Evaluating different levels

DSA

Objective

Competencies to Evaluate

Problem Solving

Objective

Competencies to Evaluate

Evaluating different levels

LLD

Objective

Competencies to Evaluate

Evaluating different levels

HLD

Objective

Competencies to Evaluate

Evaluating different levels

Bar Raiser

Objective

Competencies to Evaluate

This document outlines the standardised hiring process for Software Development Engineers (SDE1 to SDE4) in our team. The goal is to ensure consistency, fairness, and quality across the board while scaling our hiring efforts. Each candidate will go through four/ five rounds:

1. Machine Coding
2. DS and Algo
3. Problem Solving
4. Low-Level Design (LLD)
5. High-Level Design (HLD)
6. Bar Raiser
7. Hiring Manager Round

 Do not treat sample questions as a question bank, but use it more as a reference for type and difficulty of the questions. Feel free to add your own sample questions!

Level	Machi ne Coding	DSA	PS	LLD	HLD	BR	HM

SDE1	Yes	Yes	No	Yes	No	No	Yes
SDE2	Yes	Yes	Yes	Yes	No	Yes	Yes
SDE3	Yes	Yes	Yes	Yes	Yes	Yes	Yes
SDE4	No	Yes	Yes	Yes	Yes	Yes	Yes
Staff	No	Yes	Yes	Yes	Yes	Yes	Yes

Machine Coding

Objective

Primary objective of this round is to assess how well a candidate translates the solution to a fully functioning, runnable, production-ready code. Idea is not to test the candidate's DSA core skills, but to focus more on the code quality.

Competencies to Evaluate

- **Correctness of the code** - Is the code doing what it is supposed to do. There should be a list of exhaustive test cases and how many of them are passing through.
- **Code quality** - Readability of the code, testability of the code, exception handling, duplication of logic, right abstractions in place, method signatures, etc.

Evaluating different levels

The core evaluation criteria remain largely consistent across levels. However, for more senior roles, equal emphasis should be placed on code quality including readability, maintainability, and structure along with correctness.

DSA

Objective

The objective of this round is to evaluate how candidates approach concrete problem statements by selecting and implementing appropriate data structures and algorithms. This round emphasises optimisation for both time and space complexity.

Competencies to Evaluate

- **Understanding the Problem** - Is the candidate able to understand the problem with few examples and think of solutions.
- **Data structure and Algorithm selection** - Is the candidate able to recognise the pattern in the problem and chose the appropriate data structures for the same.

- **Optimisation** - Candidate may start off with brute force approach, but is he able to identify the bottlenecks and come up with an optimised algorithm/ approach. And can he justify the optimised approach scientifically with space and time complexity analysis.
- **Pseudo code quality** - Is the candidate able to translate the optimised solution to a pseudo code which covers edge cases and special scenarios.

Problem Solving

Objective

Given a fuzzy real world problem, can the candidate bring in clarity and structurally solve the problem using computer science fundamentals. Idea is to give a small business problem (not the system design type) that can be solved using a combination of data structures, data modeling and logic.

Competencies to Evaluate

Handling ambiguity - Is the candidate able to navigate the problem space by asking right question, thinking of various scenarios and having a meaningful discussion.

Understanding the system constraints - Every system will have some constraints. Is the candidate able to understand the constraints and model the solution according to that. These constraints can be space, compute, latency, etc.

Trade off analysis - Is the candidate able to explain the choices being made. Is the candidate able to explain pros and cons of multiple approaches and can justify why a particular approach is better.

Evaluating different levels

Below is an example of how we can evaluate different levels for the question ***We need to migrate 100 million user records from MySQL to MongoDB with zero downtime. How would you approach this?***

Competency	SDE2	SDE4	Staff Engineer
Handling Ambiguity	Asks basic clarifying questions like: "Is downtime truly zero?" or "Can we pause writes?" May assume bulk migration is enough.	Asks deeper questions: "Is MongoDB the new source of truth?" "Should we migrate all or only active users?"	Reframes the problem entirely: "Are we designing a general-purpose migration framework?" "How will we validate integrity

	Misses edge scenarios like ongoing writes or rollback.	“What’s the fallback if something fails?” Attempts to define boundaries and de-risk unknowns.	live?” “Should this be reversible?” Aligns technical plan with business and infra strategy.
Understanding System Constraints	Recognizes it’s a large dataset. Suggests basic data dump and restore. May not account for system load or concurrent traffic.	Understands real-time writes during migration are a challenge. Proposes full export + CDC (e.g., Debezium, Kafka). Mentions idempotency, versioning, partial failures.	Quantifies system-level impact: “At 10M rows, how much bandwidth and time?” “What’s the impact on MySQL during CDC?” “Can we throttle replication intelligently?” Designs for high availability and observability.
Trade-Off Analysis	May compare SQL dump vs manual script. Struggles to balance correctness and simplicity. Often favors “get it done” approach.	Compares different techniques: Full snapshot vs incremental sync, Dual-write vs event-based replication Explains trade-offs around data consistency, migration time, operational complexity.	Thinks in platform abstractions: “Should this support multiple data stores in future?” “What are the rollback implications at scale?” “What’s the blast radius of failure?” Includes tooling, alerting, shadow traffic, canarying, rollback, A/B validation.

LLD

Objective

Objective of this round is to check how a problem solution is translated to a well structured code using OOP fundamentals

Competencies to Evaluate

Design patterns - Does the solution use the right patterns i.e., factory pattern, chain of responsibility, observer pattern, etc.

OOP fundamentals - How are the entities defined, what are the attributes chosen, how does the value for these attribute change, how do these entities interact with each other, API design, etc.

SOLID principles - Does the solution follow single responsibility principles, DRY principles, etc.

Extensibility and maintainability of the code - How hard will it be to add adjacent new requirements? Is logging and monitoring treated as first class citizens? How are exception scenarios handled?

Evaluating different levels

Overall objective and the competencies remain the same.

Below is one example approach for the question and what to expect from each level ***design a parking lot system.***

Evaluation Area	SDE2	SDE4	Staff Engineer
1. Design Patterns	Uses basic OOP + composition . May define <code>ParkingLot</code> , <code>Vehicle</code> , and <code>Spot</code> classes. Might miss patterns like strategy or factory. Spot allocation logic hardcoded.	Applies Strategy Pattern for spot allocation (e.g., <code>NearestAvailableSpotStrategy</code> , <code>FirstComeStrategy</code>). Uses Factory Pattern to create ticket/spot types. Recognizes potential for Observer for notifications.	Designs for multi-entrance/multi-exit lots using Chain of Responsibility (entry gates). Suggests Event-driven model (Observer) for external systems like billing/logging. Pluggable pricing strategy using Strategy Pattern .
2. OOP Fundamentals	Defines basic classes like <code>Vehicle</code> ,	Entities have clear ownership and	Models system for distributed

	ParkingSpot , Ticket . Attributes are often hardcoded or mixed across layers. May not define clear interfaces.	responsibilities: Vehicle (type, plate), ParkingSpot (type, occupied), Ticket (entry time). Interfaces like IParkingStrategy and ITicketService . Thinks in modules.	ownership: Zone , Floor , Sensor , PricingPolicy , Violation . Thinks about abstracting ParkingLot as a platform — supporting multiple types (open/valet/mall/etc).
3. SOLID Principles	Understands Single Responsibility at a class level (e.g., ParkingLot does too much). May not separate concerns like fee calculation.	Applies SRP, OCP, LSP well: FeeCalculator Service , TicketService , ParkingManager are separate. Adds interfaces for substitution and extensibility.	Ensures complete modularity and loose coupling. E.g., new VehicleType or PaymentMode requires zero modification, only extension. Discusses interfaces for external systems (e.g., payment gateway, sensors).
4. Extensibility & Maintainability	Changes like adding electric spots or valet zones might require major refactoring.	Code supports extensions like new vehicle types, pricing rules, or notification mechanisms Handles edge cases like full lot, expired ticket. Logging and alerts considered.	Mentions plug-and-play modules: pricing engine, spot allocator, billing integration. Proposes config-driven behaviors (e.g., surge pricing, reserved spots). Suggests monitoring via Prometheus/log aggregation for spot usage.

HLD

Objective

Given a fuzzy real world problem, how can a candidate navigate the functional and non functional requirements and can come up with a system design that solves the problem.

Competencies to Evaluate

Understanding the Problem - Can the candidate drill down in the problem's functional and non functional requirements. Can the candidate identify few critical requirements and prioritise them and can define the scope for the design.

Problem decomposition - Can the candidate break down the problem into manageable chunk of problems and tackle them structurally by choosing right abstractions and technology.

Scalability - Can the candidate identify the bottlenecks in the system design and address them.

Disaster recovery and maintainability - Instrumentation of the system, DLQ setups for error handling, replaying events scenarios, operational excellence, etc.

Trade off analysis - Can the candidate explain the choices being made and the options that are evaluated. SQL vs NoSQL, strong consistency vs eventual consistency, offline vs online, etc.

Evaluating different levels

Overall objective and the competencies remain the same. The place where it differs is the evaluation depth of each competency and the breadth of the solution.

Below is one example approach for the question **design a system to rate-limit API requests per user across multiple services and data centres**.

Evaluation Area	SDE2	SDE4	Staff Engineer
1. Problem Exploration & Ambiguity Handling	"Can I assume that each user has a fixed request limit, like 100 req/min?"	"Are limits user-specific or can we have tiers (e.g. free vs premium)?"	"Do rate limits need to adapt dynamically (e.g., during promos or flash sales)?"
	"Is this for a single service or multiple microservices?"	"Is this system meant to work across regions — e.g., US-East and Asia?"	"Should this integrate with existing abuse prevention tools?"

2. System Constraints & Assumptions	"I'll store a per-user key like <code>user:1234</code> in Redis with an expiry of 60s. That should enforce 100 req/min."	"To handle bursty traffic, I'll use a token bucket model with capacity 100 tokens and refill rate of 1.66 tokens/sec." "I assume Redis cluster latency < 5ms, otherwise we'll hit latency issues at high QPS."	"For low-latency geo-distributed enforcement, I propose using edge-located rate-limiters (e.g., Envoy Lua filter) backed by a replicated gRPC control plane syncing quotas every 2s."
3. Solution Design & Depth	Uses Redis INCR + EXPIRE approach. "I'll call Redis on each request. If count > 100, reject. Otherwise allow."	Uses local + central enforcement: "We'll enforce soft limits locally using in-memory counters per pod, synced with a Redis cluster every second." "A Kafka stream can track rate-limiting metrics globally."	Proposes policy-as-a-service: "Each team can define custom rules via a YAML DSL (<code>limit: 200 req/60s per ip+apiKey</code>), managed centrally and pushed to local filters via xDS or gRPC push."
4. Trade-offs & Alternatives	Compares Redis vs in-memory: "Redis is persistent across pod restarts, but might be slower."	Compares Redis, in-memory, and envoy filters: "In-memory is fastest but lacks global view; Redis gives consistency but increases latency. Edge filters give the best of both with tuning."	Proposes regional edge + eventual consistency: "We accept eventual enforcement consistency across regions to reduce cross-DC traffic. That's a tradeoff between strict enforcement and low latency."

5. Operational Concerns	"We should set a TTL on Redis keys to prevent memory buildup." "If Redis is down, we can fallback to allowing all."	"We'll track <code>rate_limited_requests_total</code> and <code>limit_enforcement_latency_ms</code> in Prometheus." "We should guard against Redis hotkeys due to VIP users."	"We'll integrate into the platform's central logging and alerting system. If drop rate > threshold, alert SRE. Also, chaos tests should simulate Redis and network failures weekly."
6. Communication & Leadership	Explains each step as they go. Needs prompting to explore new areas.	Starts with high-level approach, then drives detail exploration. Says things like: "Let me outline my layers: client → ingress → limiter → backend..."	Frames solution as reusable infra: "Let's treat this as a multi-tenant platform component that other teams can plug into. Here's how I'd pitch this to product/security teams..."

Bar Raiser

Objective

Objective of bar raiser is 2 fold. First, getting to understand how deeply the candidate understands what he/she has done in the past. Second, how well he fits in the team from cultural perspective.

Competencies to Evaluate

Depth in past work - Assess the candidate's technical depth and ownership of previous projects. Evaluate their ability to explain complex technical decisions, trade-offs made, challenges overcome, and lessons learned.

Long term thinking - Evaluate the candidate's ability to think beyond immediate solutions and consider scalability, maintainability, and future implications. Assess their understanding of technical debt, system evolution, and strategic planning. Look for examples of decisions made with future growth in mind, anticipation of potential issues, and building solutions that can adapt to changing requirements.

Learn and be curious - Assess the candidate's growth mindset, intellectual curiosity, and continuous learning habits.